Calcolatori Elettronici
Esercitazione 2

M. Sonza Reorda – M. Monetti
M. Rebaudengo – R. Ferrero
L. Sterpone – M. Duni

Politecnico di Torino
Dipartimento di Automatica e Informatica

Obiettivi

- Condizioni di overflow
- Strutture di controllo
  - condizioni di salto
    - condizionato (*beq, bne, etc..)*
    - non condizionato (*j*)

Strutture di Controllo

- Queste istruzioni alterano l'ordine di esecuzione delle istruzioni:
  - La prossima istruzione da eseguire non è l'istruzione successiva all'istruzione corrente
  - Permettono di eseguire cicli e condizioni

Istruzioni di salto condizionato

- **Istruzioni di salto**: viene caricato un nuovo indirizzo nel program counter(**PC**) (invece dell'indirizzo dell'istruzione seguente)
- **Istruzioni di salto condizionato** (conditional branch): il salto viene eseguito **solo se** una certa condizione risulta soddisfatta
- Esempi: **beq** (*branch on equal*) e **bne** (*branch on not equal*)
  - beq r1, r2, L1 # go to L1 if (r1 == r2)
  - bne r1, r2, L1 # go to L1 if (r1 != r2)

Istruzioni di salto incondizionato

- **Istruzioni di salto incondizionato** (unconditional jump): il salto viene sempre eseguito.
- Esempi: **j** (*jump*) e **jr** (*jump register*) e jal (*jump and link*)
- j L1 # go to L1
- jr ra # go to address contained in $ra
- jal L1 # go to L1. Save address of next instruction in register ra

Confronto su minore/maggiore

- Spesso è utile condizionare l'esecuzione di una istruzione al fatto che una variabile sia minore (maggiore) di una altra:

slt t0, t3, t4 # ***set on less than***

se t3 < t4 assegna a t0 il valore **1** (*vero*)
altrimenti assegna a t0 il valore **0** (*falso*).

Con slt, beq e bne si possono implementare tutti i test sui valori di due variabili (==, !=, <=, >, >=)

Branch vari

`beq`	Branch ==	rs1, rs2, label
`bne`	Branch !=	rs1, rs2, label
`blt`	Branch <	rs1, rs2, label
`bgt`	Branch >	rs1, rs2, label
`bge`	Branch >=	rs1, rs2, label
`ble`	Branch <=	rs1, rs2, label
`bltu`	Branch < (U)	rs1, rs2, label
`bgtu`	Branch > (U)	rs1, rs2, label
`bgeu`	Branch >= (U)	rs1, rs2, label
`bleu`	Branch <= (U)	rs1, rs2, label

Branch vari (esempi)

- Esempio: blt t1, t2, label #salta a label se t1<t2
- Equivalente: slt t0, t1, t2 #t0=1 se t1<t2 altrimenti t0=0

  bne t0, zero, label #confronta t0 con il registro zero, se t0!=0 salta

- Esempio: ble t1, t2, label
- Equivalente: slt t0, t2, t1 #t0=1 se t2 < t1

  beq t0, zero, label #branch se t1<=t2

Esercizio 1

- Creare un programma che nel main acquisisca due interi da memoria e li salvi in due registri.
- Si esegua la somma e si verifichi mediante strutture di controllo, la condizione di overflow.
- In caso di overflow, informare l'utente stampando un messaggio di errore, altrimenti salvare in memoria il risultato.
- Suggerimento: se la somma di due operandi con lo stesso segno produce un risultato di segno opposto, c'è overflow. Utilizzare istruzioni di *slt e beq* per controllare il segno. Utilizzare i valori 1 e 0x7FFFFFFF come operandi per validare il corretto funzionamento del programma.

Esercizio 2

- Creare un programma che implementi il seguente codice ad alto livello:

```
if (i < j) f=i+j;
else f=i-j;
```

- Siano i,j valori acquisiti da tastiera e salvati nei registri.
- Stampare il valore di f.

Esercizio 3

- Creare un programma che implementi il seguente codice ad alto livello:

```
g=0
N=10
for(i=0; i <N; i++){
        g+=i
}
```

Esercizio 4

- Creare un programma che data la stringa
  «abceebceebeebbacacb»

conti l'occorrenza di un carattere.

- Si salvi la stringa in memoria
- Si assuma che il carattere da cercare sia salvato in memoria
- Suggerimento: utilizzare istruzioni beq, j per iterare tra i caratteri della stringa. Si sfrutti la presenza del carattere di fine stringa "0" per terminare l'iterazione.